

# JPA vs Hibernate vs Spring Data JPA

Hands-on Study Notes — CTS Deepskilling

## 1. Quick Definitions

| Term            | What it is                                                                                                                            |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------|
| JPA             | A specification (set of interfaces/rules) for ORM in Java. Defines how to map Java objects to database tables. Not an implementation. |
| Hibernate       | A concrete implementation (provider) of the JPA specification. Also offers extra features beyond JPA.                                 |
| Spring Data JPA | A Spring abstraction built on top of JPA. Reduces boilerplate by auto-generating repository implementations.                          |

## 2. How They Relate (Hierarchy)

Spring Data JPA → sits on top of → JPA (specification) → implemented by → Hibernate (provider)

In simple terms: JPA defines the rules, Hibernate does the actual work of talking to the database, and Spring Data JPA makes your code shorter by writing the repository code for you.

## 3. Detailed Comparison

| Aspect          | JPA                                                 | Hibernate                                                  | Spring Data JPA                                     |
|-----------------|-----------------------------------------------------|------------------------------------------------------------|-----------------------------------------------------|
| Type            | Specification (interfaces)                          | Implementation (library)                                   | Abstraction layer (framework)                       |
| Package         | javax.persistence / jakarta.persistence             | org.hibernate                                              | org.springframework.data.jpa                        |
| Provides        | Annotations & contracts (Entity, Id, EntityManager) | Actual ORM engine, HQL, caching, lazy loading              | Repository interfaces, query methods, pagination    |
| Boilerplate     | Medium (need EntityManager code)                    | Medium                                                     | Very low (just extend an interface)                 |
| Can run without | N/A                                                 | Cannot run without a JPA provider role (it implements JPA) | Needs a JPA provider underneath (usually Hibernate) |

## 4. Code Example: Same Goal, Three Layers

### Using plain JPA (EntityManager):

```
EntityManager em = emf.createEntityManager();
em.getTransaction().begin();
```

```
Student s = em.find(Student.class, 1L);
em.getTransaction().commit();
```

### Using Hibernate directly (Session API):

```
Session session = sessionFactory.openSession();
session.beginTransaction();
Student s = session.get(Student.class, 1L);
session.getTransaction().commit();
```

### Using Spring Data JPA (Repository):

```
public interface StudentRepository extends JpaRepository {
}

// Usage - no implementation code needed!
Student s = studentRepository.findById(1L).orElseThrow();
```

## 5. Suggested Hands-on Task

- Create a Spring Boot project with Spring Data JPA + H2/MySQL dependency.
- Define an Entity class (e.g., Student) using JPA annotations (@Entity, @Id, @GeneratedValue).
- Create a repository interface extending JpaRepository<Student, Long>.
- Run the app and observe Hibernate auto-generating the table (check logs for Hibernate SQL).
- Use built-in methods like save(), findById(), findAll() — notice you wrote zero implementation.
- Add a custom query method, e.g., List<Student> findByName(String name), and test it.

## 6. Quick Reference Summary

**JPA** = the rulebook (interfaces only, no code execution).

**Hibernate** = the engine that actually follows the rulebook and talks to the database.

**Spring Data JPA** = the assistant that writes the repository code for you, using Hibernate (or another JPA provider) underneath.